

Assignment 14

Garbage Collection and Circular References in Python

ARITRA TALUKDAR;CSB24034

Aim

To demonstrate circular references in Python and show how the Garbage Collector removes unreachable objects from memory.

Objectives

- To understand reference counting in Python.
- To create circular references between objects.
- To investigate unreachable objects in memory.
- To use the `gc` module for garbage collection.

Theory

Python primarily uses reference counting for memory management. Every object keeps track of how many references point to it.

When the reference count becomes zero, the object is automatically destroyed.

However, in circular references:

- Object A refers to Object B
- Object B refers to Object A

Even after deleting external references, the objects still reference each other, so their reference counts do not become zero.

Such objects become unreachable but remain in memory.

Python's Garbage Collector (`gc`) detects and removes these cyclic references using `gc.collect()`.

Algorithm

1. Create a `Node` class with:
 - name
 - link attribute
2. Create two objects:
 - Node A
 - Node B
3. Create a circular reference:
 - A links to B
 - B links to A
4. Display reference counts using `sys.getrefcount()`.
5. Delete original references using `del`.
6. Use the `gc` module to investigate whether objects still exist.
7. Call `gc.collect()` to remove unreachable objects.
8. Display the number of collected objects.

Program

```
1 import sys
2 import gc
3
4 class Node:
5
6     def __init__(self, name):
7         self.name = name
8         self.link = None
9
10    def __del__(self):
11        print(f"{self.name} is being garbage collected")
12
13 gc.set_debug(gc.DEBUG_SAVEALL)
14
15 print("GARBAGE_COLLECTION_DEMONSTRATION")
16
17 input("Press ENTER to create nodes...")
18
19 A = Node("Node_A")
20 B = Node("Node_B")
21
22 print("Nodes created successfully.")
```

```

23
24 input("Press ENTER to create circular reference...")
25
26 A.link = B
27 B.link = A
28
29 print("Circular reference created.")
30
31 input("Press ENTER to check reference counts...")
32
33 print("Reference count of A=", sys.getrefcount(A))
34 print("Reference count of B=", sys.getrefcount(B))
35
36 a_id = id(A)
37 b_id = id(B)
38
39 input("Press ENTER to delete references...")
40
41 del A
42 del B
43
44 print("Variables deleted.")
45
46 input("Press ENTER to investigate memory...")
47
48 found_a = False
49 found_b = False
50
51 for obj in gc.get_objects():
52
53     if id(obj) == a_id:
54         found_a = True
55
56     if id(obj) == b_id:
57         found_b = True
58
59 print("Node A still exists:", found_a)
60 print("Node B still exists:", found_b)
61
62 input("Press ENTER to run garbage collector...")
63
64 unreachable = gc.collect()
65
66 print("Unreachable objects collected=", unreachable)

```

Sample Output

GARBAGE COLLECTION DEMONSTRATION

Press ENTER to create nodes...

Nodes created successfully.

Press ENTER to create circular reference...

Circular reference created.

Press ENTER to check reference counts...

Reference count of A = 3

Reference count of B = 3

Press ENTER to delete references...

Variables deleted.

Press ENTER to investigate memory...

Node A still exists: True

Node B still exists: True

Press ENTER to run garbage collector...

Node A is being garbage collected

Node B is being garbage collected

Unreachable objects collected = 2

Conclusion

The program successfully demonstrated circular references in Python. Even after deleting the original references, the objects remained in memory because they referenced each other. Using `gc.collect()`, Python's Garbage Collector identified and removed the unreachable objects, preventing memory leaks.